Engineering Economics for Computer Engineers

Technical decisions become economic decisions.

TECH

+

ECON

Why Economics Matters in Computer Engineering

A technically excellent system can still be a poor engineering decision if it is too expensive to build, operate, scale, or maintain.

1. Use an LLM API or host a model ourselves?
2. Buy GPU servers or rent compute in the cloud?
3. Build a feature in-house or use SaaS?
4. Refactor legacy code or rewrite from scratch?
5. Is a 30 ms latency gain worth 2× infrastructure cost?

Project Life Cycle

From idea to operation



Example: AI customer-support assistant

Concept

What problem are we solving?
Chatbot or voice? Which channels?

Design

Cloud or local?
Which model, database, backend?

Operation

Cloud bills, API usage, monitoring, maintenance.

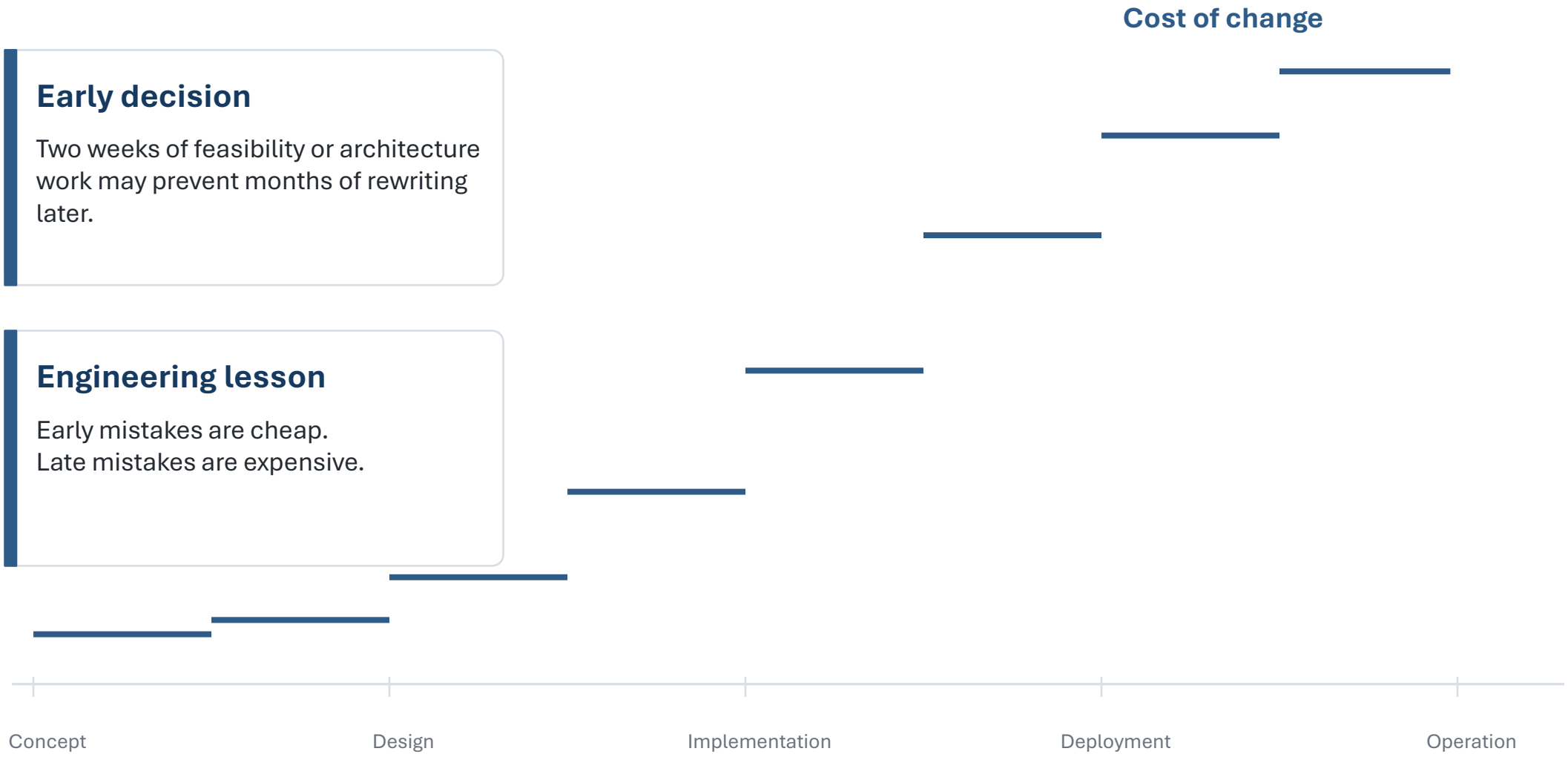
The Later You Change, the More It Costs

Early decision

Two weeks of feasibility or architecture work may prevent months of rewriting later.

Engineering lesson

Early mistakes are cheap.
Late mistakes are expensive.



Feasibility Study

Before writing code: Is this project worth doing?

Market Feasibility

Is there real demand?

Who will use it?
Will customers pay?
What alternatives already exist?

Technical Feasibility

Can we build it with available technology, skills, time, compute, security and performance constraints?

Economic Feasibility

Will benefits outweigh development, cloud, API, maintenance, staffing and risk costs?

A project can be technically possible but economically unreasonable.

Example: AI Customer-Support Assistant

Problem Thousands of repetitive support requests every month.

Possible solution LLM + company knowledge base + web/WhatsApp interface.

Market

Does support volume justify automation?
Will users accept an AI assistant?

Technical

Can the model answer accurately enough?
Can the system meet latency/security needs?

Economic

Development + API + cloud + maintenance
versus labor savings and service quality.

Engineering Economic Analysis

Evaluate costs, revenues and benefits over the life of a system—not only at launch.

Prioritize

Which project should receive limited budget?

Design

Which architecture is cost-effective?

Operate

How should we run and maintain it?

Decide

Accept, reject, lease, buy, build or outsource?

Fixed Cost vs Variable Cost

Fixed Cost

Does not change much with usage.

- GPU/server purchase
- Base developer salaries
- Annual enterprise license
- Initial software development

Variable Cost

Changes with system activity.

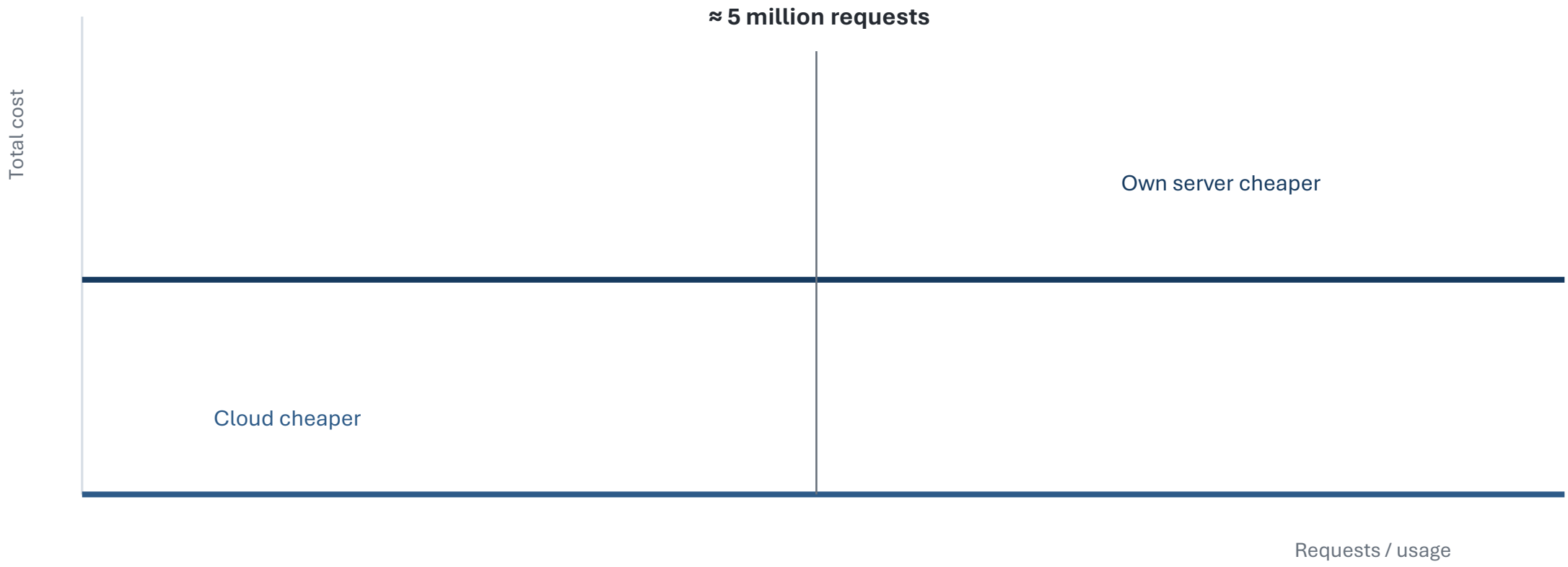
- Cloud compute hours
- API requests / tokens
- Storage usage
- Network traffic / CDN

Scale changes the best economic choice

Break-Even Example: Cloud vs Own Server

Cloud: $0 fixed + $0.01/request

Own server: $40,000 fixed + $0.002/request



Recurring and Non-Recurring Costs

Recurring Costs

Repeat regularly:

- Cloud bill
- API subscriptions
- Monitoring tools
- Database hosting
- Developer salaries

Example: $3,000/month cloud = $36,000/year

Non-Recurring Costs

Occur once or irregularly:

- Initial development
- GPU purchase
- Database migration
- Penetration testing
- Major upgrade / incident recovery

Sunk Cost: The Past Cannot Be Changed

Money already spent should not control today's decision.

Already spent on legacy platform	**$500,000**
Possible annual savings with replacement	**$250,000**

Wrong question

"But we already spent $500,000."

Better question

"From today forward, which option creates more value?"

Engineering economics focuses on present and future alternatives.

Opportunity Cost: Developer Time Is Not Free

Choosing one use of a scarce resource means giving up another.

Option A

4 developers work on an AI chatbot.
Expected profit: $150,000

Option B

The same developers build a mobile app.
Expected profit: $100,000

Resources cannot be used twice

If you choose B, the value you give up from A matters.

Build vs Buy

Build Authentication In-House

Initial engineering: $30,000
Maintenance: $1,000/month

Pros:
- Full control
- Custom integration

Risks:
- Security burden
- Maintenance
- Opportunity cost

Buy an Identity Service

Service: $800/month

Pros:
- Faster launch
- Less security engineering
- Lower maintenance

Risks:
- Vendor dependency
- Pricing at scale

Time Value of Money

$10,000 today is worth more than $10,000 three years from now.



Money today can be invested, can earn a return, and can fund engineering work immediately.

Present Value and Future Value

Present Value (P)

What future money is worth today.

Future Value (F)

What today's money will be worth in the future.

$$F = P(1 + i)^n$$

Example: $10,000 invested for one year at 10% → $11,000

Cloud Optimization: Payback Example

Optimization project cost	**$30,000**
Monthly cloud saving	**$1,500**
Annual saving	**$18,000**

Simple Payback

30,000 ÷ 18,000

≈ 1.67 years

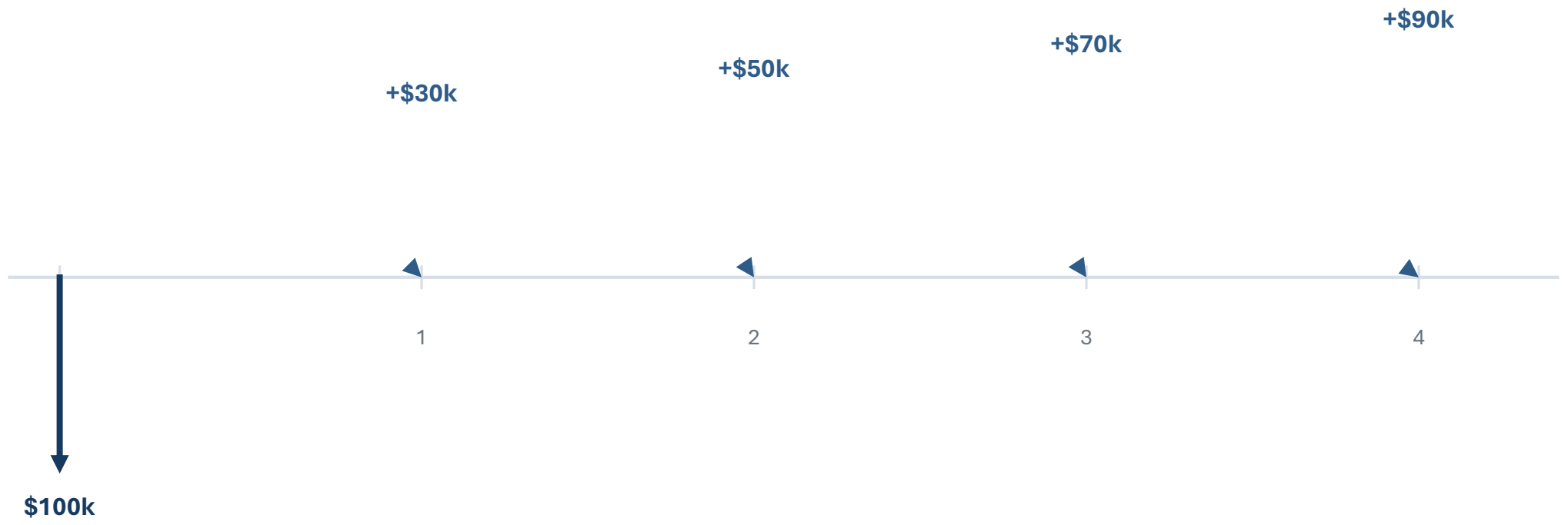
≈ 20 months

After the payback point, the savings continue.

Cash Flow Diagram for a SaaS Project

Initial development: −$100k

Net inflows: +$30k, +$50k, +$70k, +$90k



Do not simply add future cash flows—time changes their economic value.

Simple Interest vs Compound Interest

Simple Interest

Interest is calculated only on the original principal.

$F = P(1 + ni)$

$10,000 at 10% for 3 years:
$F = $13,000

Compound Interest

Interest is calculated on principal + accumulated interest.

$F = P(1 + i)^n$

$10,000 at 10% for 3 years:
$F = $13,310

Compounding becomes powerful over time

Compound Effects in Software Engineering

Small productivity improvements can generate large cumulative value.

CI/CD Improvement

Save 10 minutes
per developer per day

100 Developers

1,000 minutes
saved every day

Long-Term Effect

≈ 16.7 engineering
hours/day recovered

Automation, reusable libraries, testing infrastructure and developer tooling can create recurring economic value.

Case Study: AI Startup Infrastructure

Alternative A — Cloud API

No hardware investment

Year 1: $15,000
Year 2: $60,000
Year 3: $180,000

Nominal total: $255,000

Alternative B — Own Infrastructure

Initial investment: $120,000

Year 1: $35,000
Year 2: $40,000
Year 3: $50,000

Nominal total: $245,000

The cheaper nominal total is not automatically the better decision.

What Else Must We Consider?

Technical

Performance
Scalability
Reliability

Economic

Cost
Return
Payback

Strategic

Vendor lock-in
Flexibility
Future fit

Risk

Security
Price change
Obsolescence

Engineering economics is a decision framework—not just arithmetic.

Final Takeaway

Best Technology ≠ Best Engineering Decision

Technical Performance + **Economic Value** + Risk + Time

Engineering Decision

The real question is always: Is the technical benefit worth the economic cost?